

A: GQA6、Attention 路由与 GDN 实现和构建

本文由总览原第 4、7.1、7.2 节拆出，作为 A 的独立实现、构建和专项验收手册。全局基线、Qwen layer 命中、三人调用关系、服务冒烟和合并标准见 [Qwen3.5 优化分工总览](#)。

1. 实现

1.1 实现范围

GQA6 prefill

A 负责一个 CTA 同时计算共享 KV head 的两个 Q head。目标配置固定为 Q24/KV4/D256，grid 第二维为 `4 KV heads * 3 head pairs = 12`。短 query 使用 `BLOCK_ROWS=16, 4 warps, 2 stages`；query 长度至少 128 时使用 `BLOCK_ROWS=64, 4 warps, 1 stage`，并把 KV tile 拆成两个 32-token subtile。

kernel 必须保留标准 online softmax 状态：`max_score`、`normalizer` 和 `weighted_values`。每次读取 paged KV 后先更新最大值，用 correction 缩放旧状态，再累加当前 probabilities 和 V。causal 上界是 `context_len + local_token + 1`。

A 的 kernel 不重复检查 dtype、batch 或 cache layout；这些前置条件由 A 在宿主 backend 中的公共 gate 保证。修改地址计算或 launch shape 时，必须同步检查 gate 是否仍足够严格。

attention 公共入口与路径路由

A 负责 `RocmAiterUnifiedAttentionImpl` 中的公共 attention fast-path gate。宿主条件命中后先调用 B 的 `page784.prefill`；返回 `True` 时直接返回 output，返回 `False` 时调用 A 的 `gqa6.prefill`。只有更外层宿主 gate 不命中时才继续官方 `self.unified_attention`。

A 不进入 page784 的 pack、FlashAttention 或 LSE merge 内部，也不与 B 共享函数体。A/B 的唯一接口是 `page784.prefill(...) -> bool` 及官方已有的 tensor/metadata 参数。

GDN RMSNorm+SiLU

目标输入为 `core_attn_out=[T,48,128]`，`z.stride=(16384,128,1)`。一个 program 处理 16 行，grid 为 `T*3`。kernel 内把 x、z 和 weight 转为 FP32，完成：

```
rms = x * rsqrt(mean(x*x) + eps)
result = rms * weight * (z * sigmoid(z))
```

再按 x 的 dtype 写回。当前 helper 检查目标 layout、CUDA 和 gfx936；服务级目标 dtype 固定为 BF16。layout 或设备不匹配时，必须调用官方 `norm(x.reshape(-1,128), z.reshape(-1,128)).reshape_as(x)`。

该优化只替换 GDN output epilogue，不修改 `gd_n_attention_core` 的数学实现和 recurrent state。

A 与 B/C 的协作和解耦

- **A和B的执行关系：**二者是同一官方attention入口下的互斥fast path，不是前后相接的两段计算。A先完成公共 shape/dtype/layout/device gate，再调用B的 `page784.prefill(...) -> bool`；B返回 `True` 表示output已经完整写好，A立即返回，绝不再启动GQA6；B返回 `False` 表示尚未启动kernel、没有部分写output，A才启动GQA6。

- **A和B的接口边界**：只共享官方已有的query/current K/V/cache/output、`FlashAttentionMetadata` 和scale；不共享Triton kernel、workspace、metadata cache或内部 helper。A拥有调用顺序和公共gate，B拥有page784特有gate及其完整执行路径。
- **A和B需要同步的修改**：修改 `page784.prefill` 签名、公共cache layout/stride假设、metadata语义或bool契约时必须双方review；只调整GQA6 tile不需要B修改，只调整page784 pack/FA/merge不需要A修改。联合验收必须同时覆盖 `True` 不启动GQA6、`False` 启动GQA6、外层gate不命中走官方fallback三种路由。
- **A和C的协作**：C拥有 `gfx936.py` 文件骨架和 `is_gfx936()`；A拥有其中GDN Norm+SiLU函数段。A修改GDN公式/shape，C只做文件级合并review；C修改公共设备门，必须由A重跑attention和GDN。A的Python/Triton修改不要求重编C的native kernel，但最终版本统一使用C构建的新wheel。
- **依赖边界**：A只用官方源码已有的PyTorch和 `vllm.triton_utils`，不直接依赖或重编FlashAttention，也不增加pip包。

1.2 以官方源码为模板的改法

A不应从空文件回忆完整 attention 或 norm。先打开下面四个官方文件：

- 官方 ROCm ALTER attention 宿主
- 官方 Triton unified attention
- 官方 RMSNormGated
- 官方 Qwen3.5 model

GQA6：从官方 unified attention 做定形特化

1. 从官方 `unified_attention` 及其 2D Triton kernel 提取已有的数据流，不重新设计 attention：使用官方 `block_table` 语义、query/cache/output layout、causal 上界、FP32 online-softmax 和 BF16 output。
2. 保留官方 host 提供的 `query_start_loc`、`max_query_len`、`max_seq_len` 和 `block_table`，只把通用 `num_queries_per_kv` 特化为 Q24/KV4，即每 KV head 的 6 个 Q head 拆为 3 对。
3. 官方 kernel 已给出 `max -> correction -> denominator -> P@V` 的更新顺序；A 只把一个 Q row 改成两个共享 KV tile 的 Q row，不改变规约公式。
4. 官方实现通过 runtime stride 支持通用 cache；本版同样把 K/V 的完整四维 stride 分别传给 kernel。比赛 hybrid KV cache 是交错 view，不能从 shape 推导连续 offset；query/output 仍由宿主 gate 约束为目标连续 layout。
5. 不改官方 backend 的 decode、FP8、ALiBi、window、sink 或 output-scale 分支。专用 kernel 只由 A 维护的宿主窄 gate 调用；B 只维护 page784 的内部条件。

最小改造结构应保持为：

```
# A 在官方 forward 中判定 exact Qwen3.5 prefill gate。
if use_gqa6:
    if page784.prefill(...official metadata...):
        return output
    gqa6.prefill(...official metadata...)
    return output

# 官方 self.unified_attention(...) 调用原样保留。
```

GDN Norm+SiLU：从官方 RMSNormGated 缩成定形 kernel

1. 以官方 `RMSNormGated.forward_native` 为数值规范：FP32 计算 variance/RMS，乘 weight；`norm_before_gate=True` 时最后乘 `SiLU(z)`。
2. 以官方 `RMSNormGated.forward_cuda` 为 fallback 规范，不重写 group norm、其他 activation 或非目标 dtype 的通用分派。

- 在官方 `Qwen3_5GatedDeltaNet.forward` 中只替换 Part 3 原来的 reshape -> `self.norm(core_attn_out, z)` -> reshape，不复制 Part 1 投影、GDN core、state 或 out projection。
- helper 先判断目标 layout；不命中时执行与官方相同的 reshape/norm/reshape。fast path 内核只是官方公式在 `[T, 48, 128]` 上的定形展开。

1.3 A 的代码索引

Path	当前行号	内容介绍	可参考的官方代码（fa718036）
<code>vllm/v1/attention/ops/rocm_aiter_unified_attention_gqa6.py</code>	1-97	文件调用链说明与 <code>_gqa6_prefill</code> ：Q-head 配对、真实 K/V stride、causal mask、online softmax	<code>vllm/v1/attention/ops/triton_unified_attention.py::unified_attention</code> 及其 2D kernel
同上	100-134	<code>prefill</code> ：GQA6 block size、compiler options、grid 和 launch	<code>vllm/v1/attention/backends/rocm_aiter_unified_attn.py::RocmAiterUnifiedAttentionImpl.forward</code>
<code>vllm/v1/attention/backends/rocm_aiter_unified_attn.py</code>	22-30、103-143、228-287	文件调用链；构造期/运行期 gate；按 page784、GQA6、官方 fallback 的顺序路由	同文件官方 <code>RocmAiterUnifiedAttentionImpl.__init__/_forward</code>
<code>vllm/model_executor/layers/fla/ops/gfx936.py</code>	1-50	文件调用链、 <code>_gdn_rmsnorm_silu_gate</code> 和带官方 fallback 的 <code>qwen35_gdn_rmsnorm</code>	<code>vllm/model_executor/layers/layernorm.py::RMSNormGated.forward_native/_forward_cuda</code>
<code>vllm/model_executor/models/qwen3_5.py</code>	42、217-220	导入 helper；在 <code>Qwen3_5GatedDeltaNet.forward</code> 的 output epilogue 接管官方 norm	<code>vllm/model_executor/models/qwen3_5.py::Qwen3_5GatedDeltaNet.forward</code>

1.4 A 的验收

- GQA6 对 first-prefill、短尾 `T=16/32/64` 和长 query 分别与官方 attention 比较。
- 公共路由必须验证 page784 返回 `True` 时不再启动 GQA6，返回 `False` 时准确进入 GQA6；外层宿主 gate 不命中时继续官方 attention。
- GDN 覆盖 `T=16/32/64/128/4096`，检查 finite、max/mean error 和 allclose；目标 BF16 测试应与官方输出一致。
- CPU、非 gfx936、非目标 layout 的 GDN 必须走官方 norm；GQA 的非目标条件由 A 的 backend gate 验证。

1.5 499版与当前GQA6的同卡kernel对比

2026-08-10在DCU0/gfx936上直接比较两个Triton kernel，不经过服务、page784或attention 宿主路由。499版取自 `pra2026-bh408-repro-800` 提交 7736453 的最终DP2修正版，源文件 SHA-256为 `ab51986cac8a585ad496cd79ae3d1bfe553e78dc94a2c034f4841cb280584294`；实验代码和 调用开关不保留在本仓库。

发现连续 cache 测试会掩盖 hybrid cache 地址错误后，已重新对比。两边使用同一比赛 容器预装 Triton、同一 BF16 输入、同一 block table，并从 `[page, 2, 784, 4, 256]` storage 切出 K/V；实际 stride 为 `(1605632, 1024, 256, 1)`，与服务中的交错 view 同类。每个 shape 预热后交替计时15轮，完成3个独立进程；表中分别对三次进程中位数再取中位数。官方 FA 精度由本文件 2.2 的小矩阵 另行验证，不参与这里的kernel计时。

Query/context	499版 (ms)	当前版 (ms)	当前版相对499	实际生产路由	结论
16/4096	0.552032	0.550632	+0.254%	GQA6	当前版快
32/8192	1.089599	1.087415	+0.201%	GQA6	当前版快
64/16384	3.539134	3.520542	+0.528%	GQA6	当前版快

Query/context	499版 (ms)	当前版 (ms)	当前版相对499	实际生产路由	结论
64/32768	7.022202	6.956251	+0.948%	GQA6	当前版快
128/512	0.111072	0.110776	+0.267%	GQA6	当前版快
512/512	0.316872	0.315704	+0.370%	GQA6	当前版快
4096/512	5.431036	5.440828	-0.180%	GQA6	接近持平，499略快

所有 shape 上两版 BF16 输出逐元素相同，`cross_max_abs=0`。因此保留当前 GQA6：它在生产相关的短尾长上下文快约 0.20%--0.95%，仅 4096/512 慢 0.18%。499 版虽然物理源码略短，但同时处理多 sequence、query/output runtime stride、page64/page784 和更多调度分支；当前版把 batch 和 Q/O layout 交给宿主窄 gate，只保留完整 K/V stride。因此 499 版的闭卷实现与验证难度更高，不能仅按文件行数判断。

2. 构建与专项测试

2.1 完整 wheel 构建与隔离安装

A 使用下面这组已经在当前比赛容器执行成功的命令。构建放在临时 worktree，是因为 OpenDAS `setup.py` 会重写源码树中的 `vllm/version.py`；`--target` 安装不会覆盖容器依赖。

```
cd /public/home/tangyu408/Qwen_DCU_Worker_0/pr2026-bh408-gqa-page784-k5120
git worktree add --detach /tmp/qwen35-build-source HEAD
cd /tmp/qwen35-build-source
mkdir -p /tmp/qwen35-build/{dist,bdist,cache}

VLLM_TARGET_DEVICE=rocm MAX_JOBS=16 python3 setup.py \
  build --build-base /tmp/qwen35-build/build \
  bdist_wheel --bdist-dir /tmp/qwen35-build/bdist \
  --dist-dir /tmp/qwen35-build/dist

python3 -m pip install --no-deps --target /tmp/qwen35-build/site \
  /tmp/qwen35-build/dist/vllm-*.whl
```

后续命令统一使用：

```
REPO_ROOT=/public/home/tangyu408/Qwen_DCU_Worker_0/pr2026-bh408-gqa-page784-k5120
ARTIFACT_ROOT=/tmp/qwen35-build
SITE_DIR=/tmp/qwen35-build/site
CACHE_ROOT=/tmp/qwen35-build/cache
cd "$REPO_ROOT"
```

2.2 GQA6、路由和 GDN 专项检查

A 的 attention/GDN 都是 Python/Triton；没有单独 AOT 编译命令，第一次目标输入会 JIT。最短源码检查与同输入测试：

```
ruff check vllm/v1/attention/ops/rocm_aiter_unified_attention_gqa6.py \
  vllm/v1/attention/backends/rocm_aiter_unified_attn.py \
  vllm/model_executor/layers/fla/ops/gfx936.py \
  vllm/model_executor/models/qwen3_5.py
python3 -m py_compile vllm/v1/attention/ops/rocm_aiter_unified_attention_gqa6.py \
  vllm/v1/attention/backends/rocm_aiter_unified_attn.py
mkdir -p "$CACHE_ROOT/a-triton"
CHECK_SCRIPT="$PWD/docs/cscv/verify_qwen35_optimizations.py"
RUN_DIR="$(mktemp -d /tmp/qwen35-a-check.XXXXXX)"
(cd "$RUN_DIR" && HIP_VISIBLE_DEVICES=0 \
```

```
TRITON_CACHE_DIR="$CACHE_ROOT/a-triton" PYTHONPATH="$SITE_DIR" \  
python3 "$CHECK_SCRIPT" gqa6 gdn)
```

非代码步骤：使用新 Triton cache 冷跑一次目标 shape，再用同一 cache 热跑；首轮 JIT 不得计入性能。当前脚本覆盖 GQA6 的4组query/context长度、额外1组交错cache、GDN的 T= 16/32/64/128/4096，以及非目标stride回退。A不需要重编FlashAttention，但最终仍必须重新构建并安装完整 wheel。

3. 联合验证入口

完成本文件的专项检查后，按 [总览第 7.5 节](#) 继续执行同一新 wheel 的 DP1/DP2 小数据服务冒烟；实测版本、结果、合并顺序和故障备注 统一保留在总览第 7.6--7.8 节。